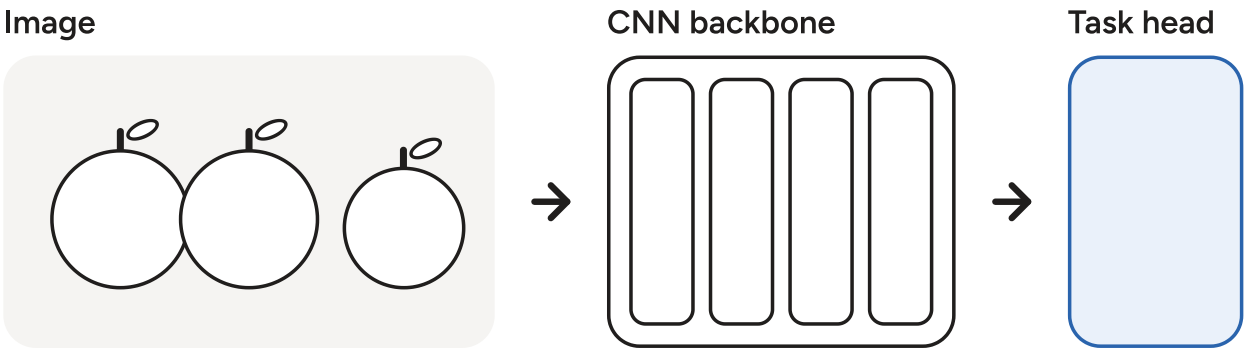
Week 5

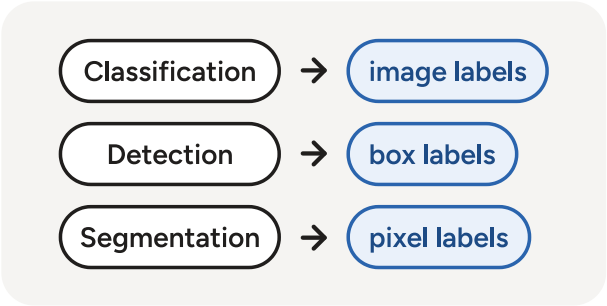
CNN applications and pretrained models

CSCI1409 Deep Learning · Lecture 1 of 2



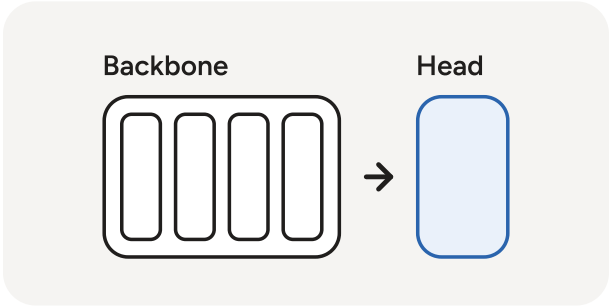
Lecture 1 learning outcomes

1



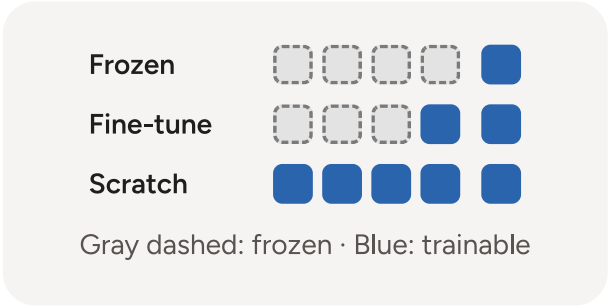
Match vision tasks to outputs and annotations

2



Explain architecture, weights, backbone, and head

3



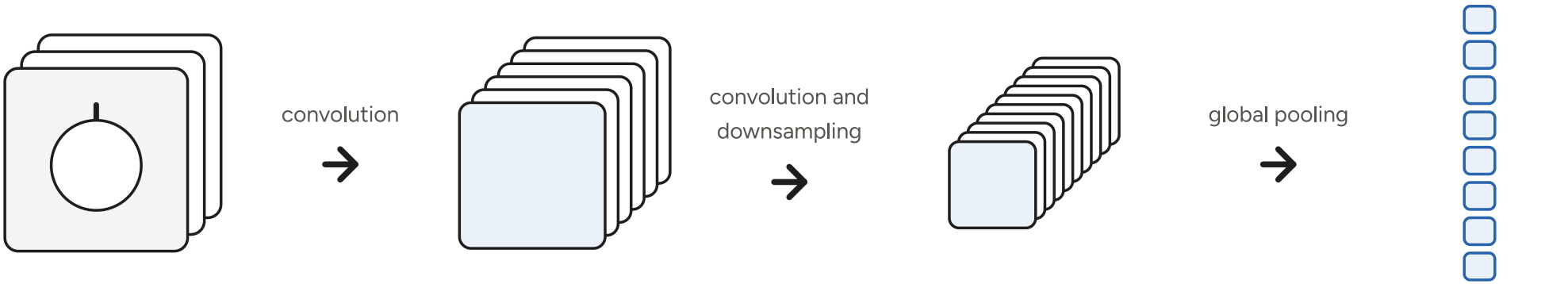
Justify a starting transfer strategy

CNN diagnostic

Write 1 min · Discuss 3 min

1. What does a feature map represent?
2. Why does the output depend on the class count?
3. Does high training accuracy prove generalization?

CNN features and pooling



RGB image

$H \times W \times 3$

Feature maps

$H_1 \times W_1 \times C_1$

Smaller feature maps

$H_2 \times W_2 \times C_2$

Pooled vector

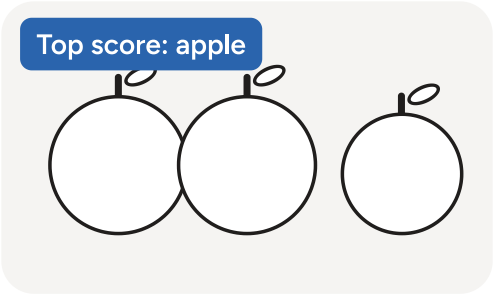
C_2 values

Schematic: exact sizes depend on the architecture

- Shared kernels detect patterns across locations
- Successive layers combine spatial evidence
- Global pooling summarizes feature channels

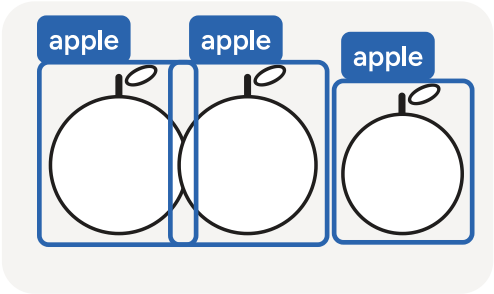
Vision tasks and required outputs

Classification



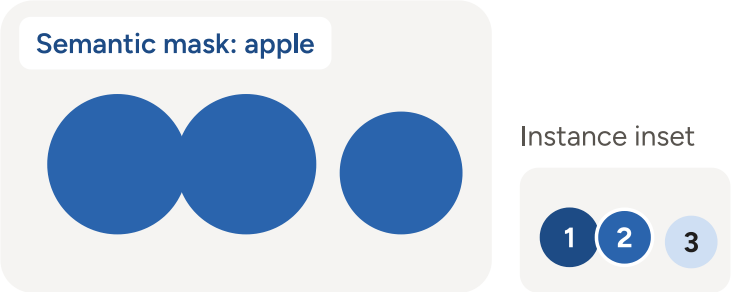
Classification gives image-level scores

Detection



Detection gives boxes and labels

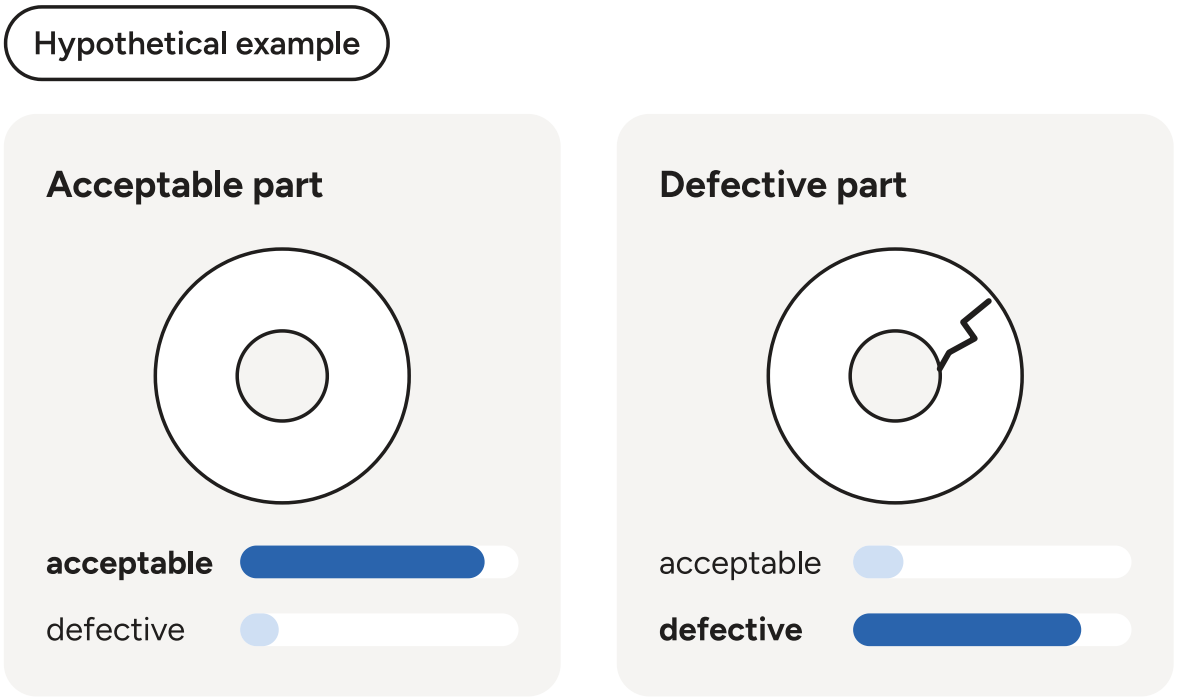
Segmentation



Segmentation gives pixel masks

Image classification applications

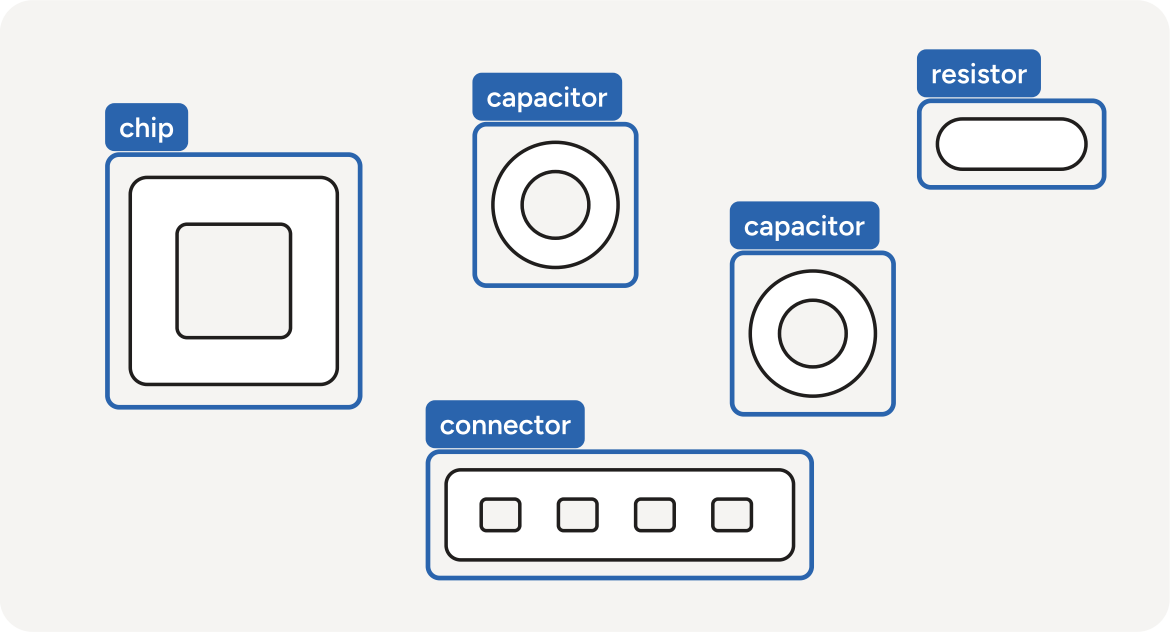
- Quality inspection can flag a photograph
- A classifier returns one score per target class
- Image labels supervise the classifier



Output: one score per class for the whole image

Object detection applications

- Locate each target object
- Predict a box and class for each detection
- Box annotations support localization



Example architecture: Faster R-CNN Schematic component scene

Semantic and instance segmentation

- Semantic masks label pixel classes
- Instance masks distinguish separate objects
- Spatial predictions need spatial annotations

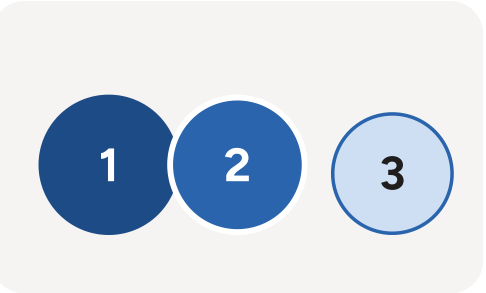
Semantic mask



apple class

background class

Instance masks



1, 2, 3 separate apple instances

touching apples stay distinct

Example semantic architecture: U-Net

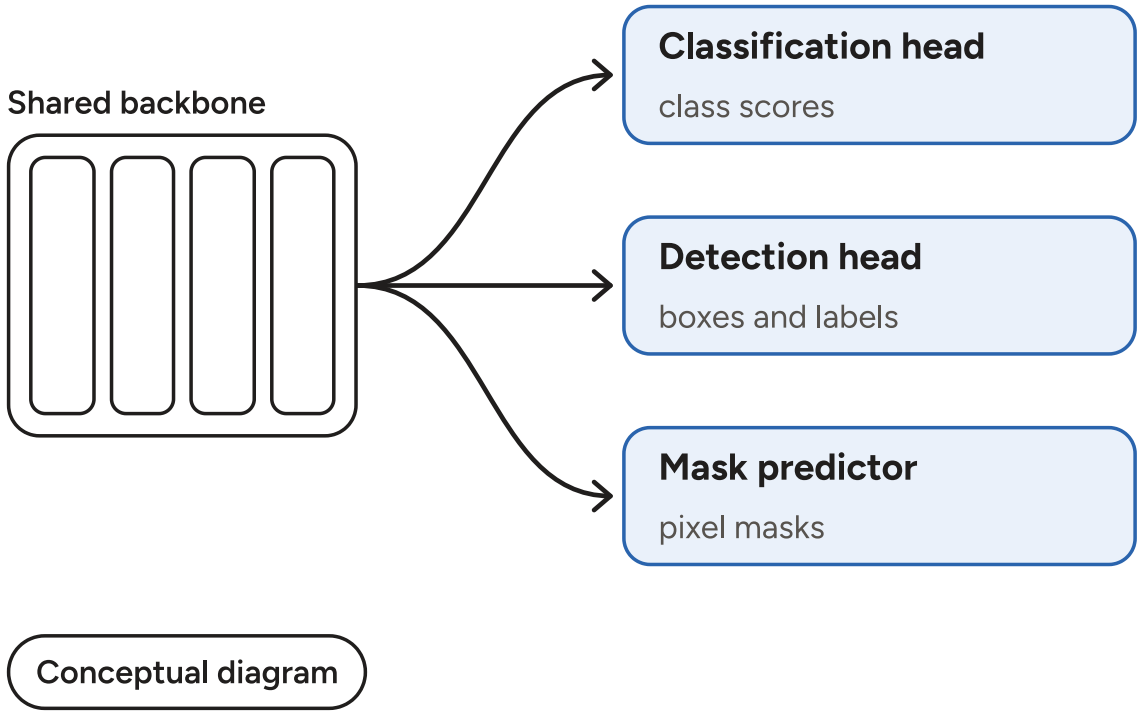
Application matching activity

Pairs · Work 2 min · Discuss 3 min

1	Flag images containing damage	Output	Annotation
2	Locate every damaged component	Output	Annotation
3	Estimate the damaged surface area	Output	Annotation

Backbone and task-specific heads

- The backbone extracts representations
- The head produces the required output
- Labels and loss must match the task



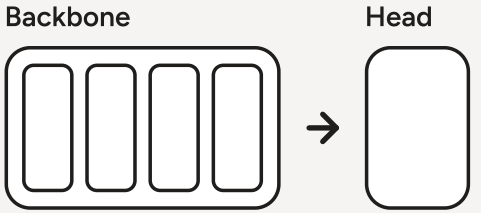
Conceptual diagram

Architecture and pretrained weights

- Architecture defines the computation
- Weights contain learned numerical values
- A checkpoint stores learned state

Random weights

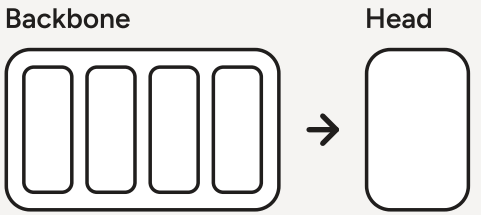
Values set by random initialization



Pretrained weights

Values learned on ImageNet-1K

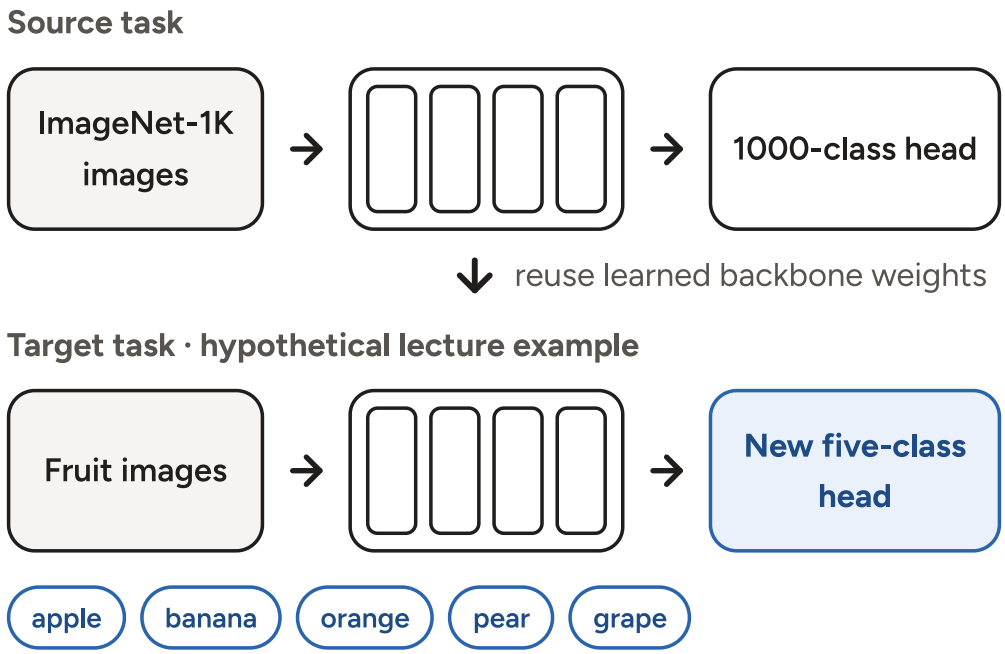
Checkpoint: parameters + buffers



Same architecture, different starting values

ImageNet pretraining and the target task

- ImageNet-1K weights learn a source classification task
- Target labels can require a different head
- Target performance still needs evaluation



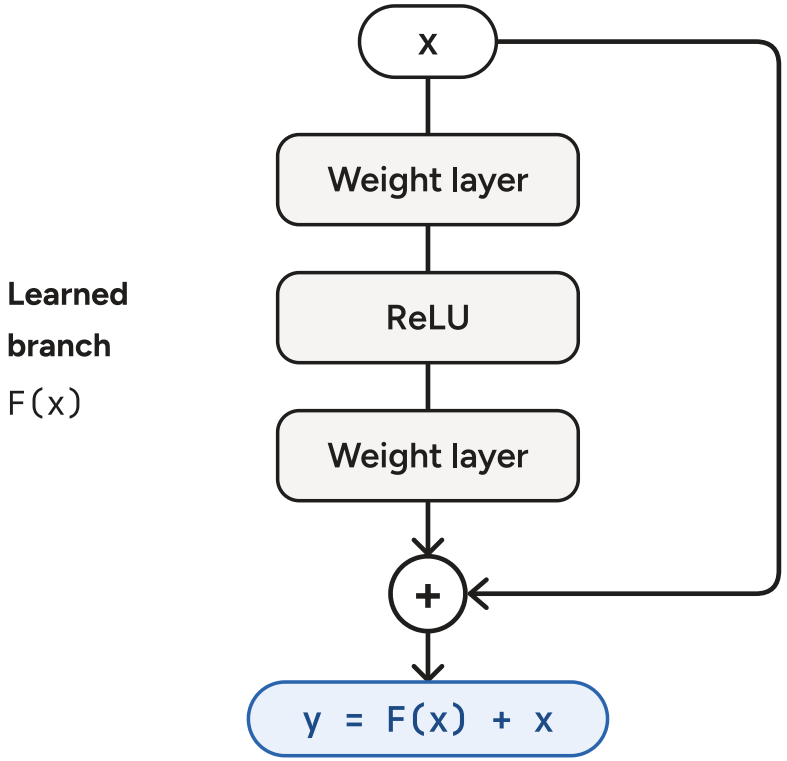
Representative CNN model families

Family	Main idea	Selection consideration
VGG	Uses a simple convolutional stack	Easy to read; check memory and compute
ResNet	Adds residual connections	Familiar backbone with a replaceable classifier
MobileNetV2	Inverted residual blocks with depthwise convolutions	Efficient candidate when resources are limited
EfficientNet	Scales depth, width, and resolution together	Efficient candidate; compare exact variants

No ranking implied: measure latency and memory on the intended device

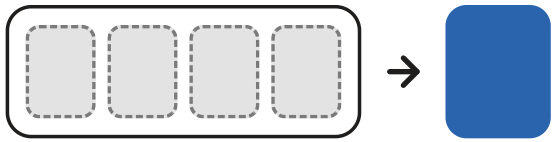
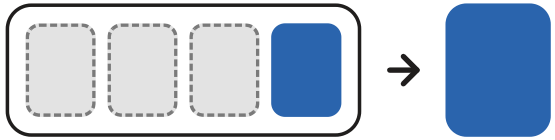
Residual connections

- A learned branch adds a correction
- The shortcut supplies a compatible representation
- A projection can match changed shapes



Transfer learning strategies

Gray, dashed: frozen Blue, solid: trainable

Strategy	Backbone → head	Initialization and updates
Frozen features train a new target head		Init: pretrained **Updates: new head only**
Fine-tuning updates selected pretrained layers		Init: pretrained **Updates: later block and head**
Training from scratch starts with random weights		Init: random **Updates: all layers**

Choosing a starting strategy

- Consider target data size and source similarity
- Begin with a useful baseline
- Adapt further when validation supports it

Starting experiments to test, not fixed rules

Related, small dataset

First experiment

→ Frozen features and a new head

Related, larger dataset

First experiment

→ Baseline, then fine-tune more layers

Different image domain

First experiment

→ Consider a domain-suitable checkpoint

Resource-constrained deployment

First experiment

→ Efficient architecture, measured on the device

Strategy selection activity

Pairs · Discuss 2 min · Feedback 2 min

Choose trainable layers and a selection rule

Scenario A

300 labelled RGB fruit images

First experiment

Next experiment

Validation rule

Scenario B

30,000 images from deployment cameras

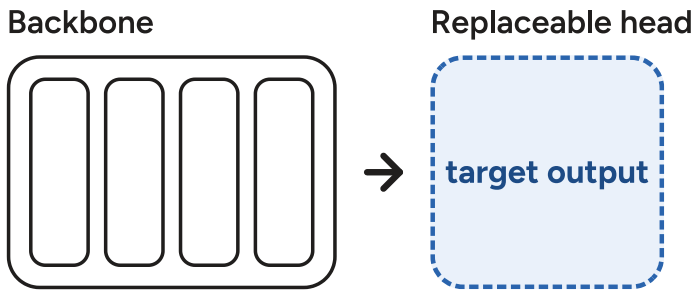
First experiment

Next experiment

Validation rule

Lecture 1 recap

1. The required output determines the task
2. Pretrained weights and frozen weights differ
3. The next lecture adapts and evaluates a classifier



Next: five-class ResNet18 example